

Assignment 1

Abgabe bis zum 12. Mai 2026 um 23:59 Uhr

Geben Sie Ihr Assignment bis zum 12. Mai 2026 um 23:59 Uhr auf folgender Webseite ab:

<https://assignments.hci.uni-hannover.de>

Ihre Abgabe muss aus einer einzelnen zip-Datei bestehen, die den Quellcode und ggf. ein PDF-Dokument bei Freitextaufgaben enthält. Beachten Sie, dass Dateien mit Umlauten im Dateinamen nicht hochgeladen werden können. Entfernen Sie die daher vor der Abgabe die Umlaute aus dem Dateinamen. Überprüfen Sie nach Ihrer Abgabe, ob der Upload erfolgreich war. Bei technischen Problemen, wenden Sie sich an Ole Brenner. Es wird eine Plagiatsüberprüfung durchgeführt. Gefundene Plagiate führen zum Ausschluss von der Veranstaltung. In dieser Veranstaltung wird ausschließlich die Java/JavaFX Version 17 verwendet. Code und Kompilate anderer Versionen sind nicht zulässig. Ihre Abgaben sollen die vorgegebenen Dateinamen verwenden. Wiederholter Verstoß gegen diese Regel kann zu Punktabzügen führen.

Aufgabe 1: Conway's Game of Life

In dieser mehrteiligen Aufgabe werden Sie verschiedene Varianten von Conways Spiel des Lebens implementieren. Die Regeln können Sie hier nachlesen: https://de.wikipedia.org/wiki/Conways_Spiel_des_Lebens. Die Aufgaben bauen aufeinander auf. Es wird jede Woche eine Musterlösung hochgeladen, die Sie verwenden können, falls Sie den vorherigen Teil nicht bearbeitet haben. Sie dürfen alte Programmteile aus den Vorwochen kopieren und wiederverwenden. Ab Woche 2 wird eine zusätzliche grafische Oberfläche zur Verfügung gestellt.

a) Erstellen Sie im Package "component" ein Interface mit dem Namen "CellRule". Implementieren Sie in dem Interface eine default-Methode `default CellType nextCellType(CellType,int)`. Die Methode hat folgende Funktionalität. Übergeben wird der Zellzustand der betrachteten Zelle sowie die Anzahl an lebenden Nachbarn, die diese Zelle umgeben. Zurückgeben soll die Methode den Zustand der Zelle in der nächsten Generation. Implementieren Sie diese Methode mit den Standardregeln.

b) Erstellen Sie in "GameOfLife" einen neuen Konstruktor `public GameOfLife(int, CellRule)`, dieser ist analog zu `public GameOfLife(int)`, mit der Ausnahme, dass dieser ein Regelwerk übergeben bekommt und in der Klasse speichert.

c) Erstellen Sie eine neue Methode `public void calculateNextGeneration(CellRule)`, die analog zur alten Methode funktioniert. Allerdings wird zur Entscheidung des nächsten Zellzustands "CellRule" verwendet. Hinweis: Sie können Ihre Implementierung von `calculateNextGeneration()` wiederverwenden, um dies umzusetzen.

Ändern Sie anschließend `calculateNextGeneration()`, sodass diese `calculateNextGeneration(CellRule)` mit den Standardregeln aufruft. Hinweis: Dies ist zum Beispiel mit `CellRule defaultRule = new CellRule(){};` möglich.

d) Erstellen Sie ein neues Package "conway.rules". In diesem soll eine Klasse "Maze" erstellt werden, die Ihr Interface implementiert. Überschreiben Sie Ihre Methode mit folgendem neuen Regelwerk:

- Eine tote Zelle lebt in der nächsten Generation, wenn diese 3 lebende Nachbarn hat.
- Eine lebende Zelle lebt in der nächsten Generation, wenn diese 1 bis 5 (inklusive) lebende Nachbarn hat.

e) Erweitern Sie Ihre Main-Methode, sodass wenn der Kommandozeilenparameter "-gui" angegeben ist, nun auch neben der Größe ausgesucht werden kann welches Regelwerk verwendet werden soll. Die Optionen sind: "normal", "seeds", "maze". Fügen Sie diese Auswahlmöglichkeit auch für die Kommandozeilen-Variante hinzu. Versuchen Sie doppelten Code zu vermeiden.

Aufgabe 2: Schatzsuche

Innerhalb dieser Aufgabe werden Sie einen endlichen deterministischen Automaten implementieren. Endliche Automaten bestehen aus einer endlichen Menge an Zuständen (States). Abhängig vom aktuellen Zustand geht man bei einer Eingabe (nachfolgend Aktion) entweder in einen anderen Zustand über oder bleibt im selben Zustand. Deterministisch bedeutet, dass für jede Eingabe im aktuellen Zustand ein eindeutiger Folgezustand erreicht wird. Endliche Automaten haben einen Startzustand und ggf. sogar mehrere Zielzustände. Ziel ist es, den in Figure 1 gezeigten Automaten zu realisieren. States werden durch Inseln repräsentiert, Pfeile stellen Transitionen dar. Es sind in jedem State die Eingaben A und B möglich. Nicht eingezeichnete Pfeile sind Transitionen, die auf denselben Zustand verweisen.

a) Implementieren Sie den Enum-Typen *Action*, welcher die Werte "A" und "B" annehmen kann. Erstellen Sie ebenfalls die Methode `public char str()`, welche eine eindeutige Repräsentation des Wertes zurück gibt.

b) Erstellen Sie das Interface *State*, welches die Methoden `public State transition(Action action)`, `public String str()` und `public String info()` definiert. Die Methode *transition*, soll die Transition abhängig vom aktuellen Zustand und der übergebenen Aktion realisieren. Die Methode gibt den Folgezustand zurück.

info() gibt die Bezeichnung des aktuellen Zustands sowie die Übergangsmöglichkeiten (z.B. Aktion A -> Shipwreck Bay; Aktion B -> Musket Hill) als String zurück. Sie können selbst entscheiden, ob Sie diese als *default*-Methode in *State* oder in der später zu erstellenden Klasse *Island* implementieren. Hinweis zur *default* Variante: Sie können *transition(Action)* und *str()* auch im Interface aufrufen, wenn noch keine konkrete Implementierung vorliegt. *str()* soll lediglich die Bezeichnung des jeweiligen Zustands zurückgeben.

Ergänzen Sie für **alle Methoden** in *State* passende JavaDoc Kommentare. Für alle anderen Methoden und Klassen ist dies nicht erforderlich.

c) Erstellen Sie zusätzlich den Enum-Typen *Island*, welcher die Inseln in Figure 1 repräsentiert. Erstellen Sie für jeden Zustand einen entsprechenden Enum-Wert. Jeder Zustand hat einen Zustandsnamen (Inselnamen). Implementieren Sie für jeden Enum-Wert die *transition*-Methode individuell. Hinweis: Eine derartige Implementierung kann wie folgt aussehen:

```

1 public enum Example implements ExampleInterface {
2     VALUE_1(String name) {
3         @Override
4         public void exampleMethodInInterface(int value) {
5             ...
6         }
7     },
8
9     ...
10
11     Example(String name) {
12         ...
13     }
14     ...
15 }

```

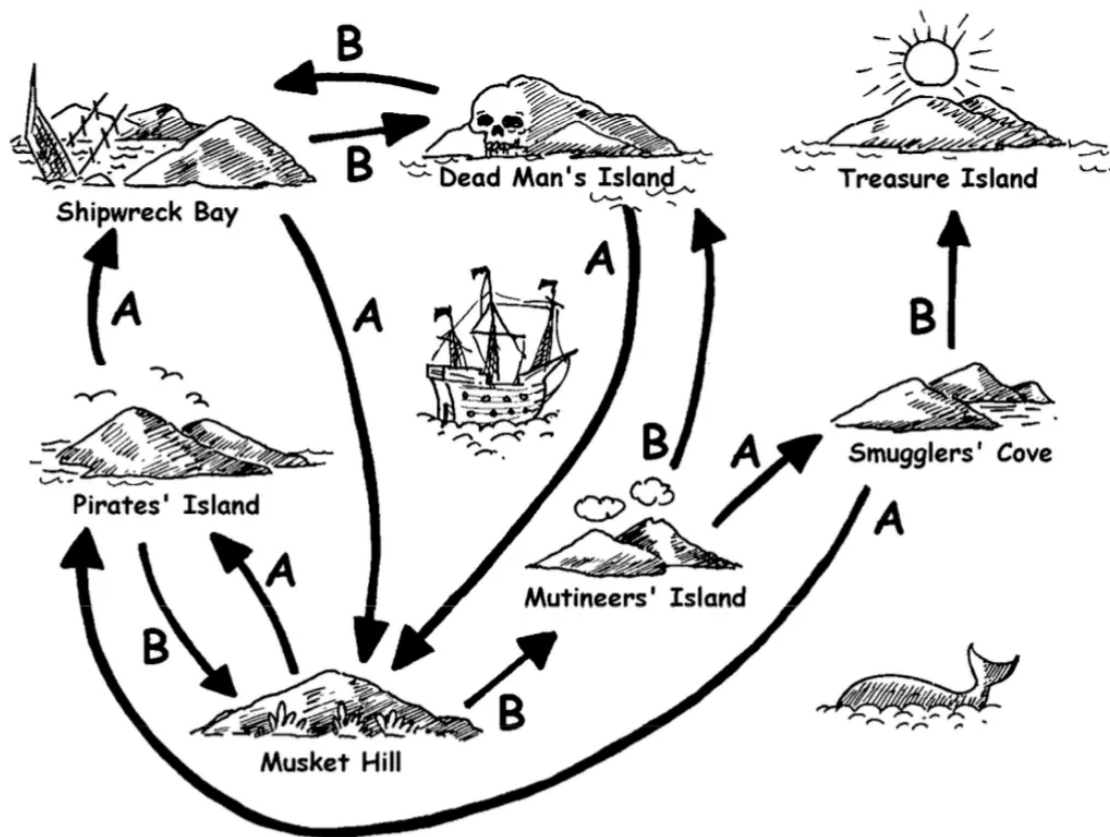


Figure 1: Endlicher Automat: States werden durch Inseln repräsentiert. Pfeile stellen Transitionen dar. Es sind in jedem State die Eingabe A und B möglich. Nicht eingezeichnete Pfeile sind Transitionen, die auf denselben Zustand verweisen. Quelle

d) Implementieren Sie die Klasse *StateMachine*, welche den kompletten Automaten darstellt. *StateMachine* speichert den aktuellen Zustand und ob der Zielzustand erreicht wurde. Der Startzustand lautet *Pirates' Island*, der Zielzustand *Treasure Island*. Ergänzen Sie eine main-Methode innerhalb derselben Klasse um ein Kommandozeileninterface, welches den Nutzer so lange nach der Wahl zwischen den Aktionen "A" und "B" fragt, bis der Zielzustand erreicht wurde. Nach jedem Zustandsübergang soll die Info des aktuellen Zustands ausgegeben werden. Wurde der Zielzustand erreicht, soll der komplette bewältigte Weg vom Startzustand bis zum Endzustand ausgegeben werden.

Aufgabe 3: Debugging

In diesen Aufgaben werden Sie einen fehlerhaften Java Codeabschnitt bekommen. Diese Fehler können **syntaktisch** oder **semantisch** sein. Es kann sich dabei um Compiler- oder Laufzeitfehler handeln. Sie dürfen den Code kompilieren und ausführen um die Fehler zu finden. Arbeiten Sie in der Template-Datei *Debug5.zip*. Diese finden Sie im Stud.IP.

a) Der Codeabschnitt enthält 5 Fehler darunter auch fehlende Methoden. Kompilieren und führen Sie den Code aus. Suchen Sie anhand der Fehlermeldungen des Compilers oder der Laufzeitfehlermeldungen Fehler im Code und korrigieren Sie diese. Kommentieren Sie am Ende der jeweiligen Zeile, was Sie korrigiert haben.

Erstellen Sie während der Fehlerkorrektur einen Blockkommentar unter dem Code, in dem Sie die Fehler dokumentieren. Schreiben Sie die Zeile(n) des Fehlers auf und beschreiben Sie den Fehler. Kopieren sie die Fehlermeldung, sofern es eine gab, anhand welcher Sie den Fehler erkannt haben. Die Dokumentation soll wie in folgendem Beispiel aussehen:

```
1 public class Debug { //K: class falsch geschrieben (ckass)
2
3   ...
4
5   /*
6   ...
7   Zeile 1: class Keyword falsch geschrieben (ckass):
8   Fehlermeldung:
9   *****
10  Debug.java:1: error: class, interface, or enum expected
11  public ckass Debug {
12      ^
13  *****
14  Der Compiler erwartet eines der drei oben angegebenen Keywords, hat aber nur das falsch
15  ↪ geschriebene ckass bekommen.
16  ...
17  */
```

Im Code vorgegebene Kommentare beschreiben immer das **korrekte** Verhalten des Programms.